

DEITIES & DEMIGODS

DUNGEON MASTER'S HANDBOOK

The Arcane Machinery Behind the AI Dungeon Master

A Technical Reference for Game Masters, Developers,
and Anyone Curious About the Code Beneath the Myth

Powered by Gemini 2.5 Flash
with Optional Groq Ultra-Low Latency Fallback

Version 1.0 | Confidential Technical Document
Generated by Z.ai

Table of Contents

Chapter 1	AI DM Architecture	3
Chapter 2	System Prompt Construction	6
Chapter 3	DM Response Schema	9
Chapter 4	State Update Protocol	12
Chapter 5	Success Rate Engine	14
Chapter 6	Narrative Voice Guidelines	17
Chapter 7	Fallback System	19
Chapter 8	Test of Faith Engine	22
Chapter 9	Difficulty Scaling	25
Chapter 10	HP Validation & Combat Safety	27
Chapter 11	Item System Internals	29
Chapter 12	Quest System	31
Chapter 13	Save/Load Architecture	32

Chapter 1: AI DM Architecture

The AI Dungeon Master at the heart of Deities & Demigods is not a simple chatbot, nor is it a basic text-completion engine. It is a **narrative state machine** -- a carefully orchestrated system that combines large language model generation with deterministic game logic, state management, and structured data validation. Every turn the player takes triggers a multi-stage pipeline that transforms a game state snapshot into a rich narrative experience while simultaneously advancing the mechanical underpinnings of the campaign. Understanding this architecture is essential for anyone who wants to modify the game, debug unexpected behaviors, or extend the system with new mechanics.

The system is powered primarily by **Google Gemini 2.5 Flash**, chosen for its ability to generate long-form narrative prose while simultaneously producing structured JSON output. The Gemini model serves as the creative engine, responsible for all storytelling, character portrayal, and dramatic narration. An optional secondary engine, **Groq**, can be configured for ultra-low-latency inference on specific sub-tasks. Groq serves as a performance accelerator rather than a primary narrative generator, and the system gracefully falls back to Gemini if Groq is unavailable or rate-limited.

1.1 Turn Processing Pipeline

Every player action triggers a deterministic seven-step pipeline. This pipeline ensures that the AI receives complete, accurate context and that the resulting narrative is mechanically consistent with the current game state. No step is skipped, and every step produces verifiable output that feeds into the next stage.

- 1. Action Selection:** The player selects an action from the dynamically generated option list. This action string is captured and passed as the primary user input to the AI DM.
- 2. State Snapshot Assembly:** The engine calls `buildDMSystem(gs)`, which serializes the entire `GameState` object into a comprehensive prompt. This includes party HP, active NPCs, quest status, prophecy state, conversation history, and all other mutable game data.
- 3. System Prompt Construction:** The serialized state is formatted into a structured system prompt containing nine distinct data blocks (detailed in Chapter 2). The prompt ends with a complete JSON schema that the AI must follow.
- 4. LLM Generation:** Gemini 2.5 Flash receives the system prompt and generates a response. The response contains both narrative prose and a structured JSON block with mechanical updates.
- 5. JSON Extraction:** A regex-based parser extracts the JSON block from the AI response. The parser handles various formatting edge cases including markdown code fences, trailing commas, and partial responses.
- 6. State Application (applyMechanics):** The extracted JSON is parsed and validated. HP deltas are applied, conditions added or removed, death flags processed, and items distributed. Every update is validated and clamped to prevent invalid states.
- 7. UI Update:** The narrative text is displayed to the player, the success rate is recalculated, new action options are generated via Groq (or fallback), and the updated game state is saved.

1.2 Gemini 2.5 Flash Configuration

The Gemini integration is configured with carefully tuned parameters that balance narrative quality against response latency and token usage. The configuration differs between opening scene generation and regular turn processing to optimize the player experience at each stage.

Parameter	Opening Scene	Regular Turn
Model	gemini-2.5-flash	gemini-2.5-flash
Max Output Tokens	8,000	4,000
Temperature	0.9	0.9
Top-P	0.95	0.95
Retry Count	3	3
Backoff Strategy	Exponential (6s, 12s, 24s)	Exponential (6s, 12s, 24s)

Table 1: Gemini API Configuration Parameters

The opening scene receives a larger token budget (8,000) because it must establish the setting, introduce characters, present the Shard artifact, and create narrative momentum. Regular turns operate at 4,000 tokens, which is sufficient for 2-4 rich narrative paragraphs plus the structured JSON response block. The temperature of 0.9 provides creative variety while maintaining enough determinism for consistent character portrayal.

1.3 Groq Fallback Configuration

Groq provides an ultra-low-latency inference option that can dramatically reduce response times for action option generation and other sub-tasks. The Groq free tier offers generous limits that make it practical for extended play sessions without cost.

Parameter	Value
Free Tier Rate Limit	30 requests/minute
Free Tier Daily Limit	14,400 requests/day
Typical Latency	50-200ms (vs. 1-3s for Gemini)
Fallback Behavior	Graceful degradation to Gemini on any error
Primary Use Case	Action option generation, summarization

Table 2: Groq API Free Tier Parameters

1.4 Throttling System

To prevent API abuse and stay within free tier limits, the engine implements a multi-layer throttling system. This system operates transparently and does not require player intervention.

- **Rate Limit:** Maximum 15 requests per minute to the primary Gemini endpoint.
- **Minimum Interval:** 1.5 seconds enforced between consecutive API requests.
- **Cooldown:** On receiving a 429 (rate limit) response, the engine enters a 15-second cooldown period before retrying.
- **Exponential Backoff:** Failed requests are retried with delays of 6s, 12s, and 24s.
- **Circuit Breaker:** After 3 consecutive failures, the engine falls back to template-based responses until the next player action.

Chapter 2: System Prompt Construction

The `buildDMSystem(gs)` function is the architectural backbone of every AI DM interaction. Called at the start of every turn, it constructs a comprehensive system prompt by serializing the current `GameState` into nine distinct data blocks. Each block provides the AI with a specific category of information, ensuring the DM has complete awareness of the campaign state without overwhelming the context window. The resulting prompt is the single most important factor in narrative quality and mechanical consistency.

2.1 The Nine Data Blocks

- 1. Party State:** A complete roster of all living player characters, including current HP, maximum HP, Armor Class, alignment, active injuries, personality traits, ability scores, class levels, and any special conditions. Dead characters are excluded from this block to prevent the AI from referencing them as active participants.
- 2. Shard Info:** The name, pantheon association, origin lore text, remaining charges, list of previously summoned entities, and current darkened status. This block gives the AI the context needed to narrate Shard-related events and summoning outcomes.
- 3. Antagonist:** Current HP and maximum HP, combat phase, banishment status, rival information (relevant in Act III), and the count of clues revealed about the antagonist identity. This block expands significantly as the campaign progresses toward the final confrontation.
- 4. Prophecy:** The current prophecy riddle (truncated to first 150 characters to conserve tokens), its state (dormant, awakening, manifesting, fulfilled, or broken), and the ID of the PC currently carrying the prophecy. The prophecy system is central to the narrative arc and mechanical bonuses.
- 5. Active Quests:** All quests currently in the "active" state, with their uncompleted objectives listed. Completed and failed quests are excluded from this block to keep the prompt focused.
- 6. Act Context:** The current act number (I, II, or III), turn limits for the current act, and the `act2StartTurn` value. This allows the AI to pace the narrative appropriately and foreshadow impending transitions.

7. Conversation History: The last 10 exchanges between the player and the AI DM, providing persistent NPC memory and narrative continuity. This sliding window ensures the AI remembers recent events without exceeding context limits.

8. Journey Summary: A full campaign TLDR that is updated each turn by the AI itself. This summary must remain under 150 words and captures the essential plot progression, key decisions, and party status. It serves as long-term memory when the conversation history window is insufficient.

9. Success Rate: The current overall success percentage along with a complete breakdown of all bonus factors contributing to it. This visibility allows the AI to calibrate narrative tension and adjust difficulty cues in the prose.

2.2 ASCII-Only Conversion

Before being sent to the AI, the entire prompt undergoes an ASCII sanitization pass via the `toAscii()` utility function. This conversion strips smart quotes (replacing them with straight quotes), em dashes (replaced with double hyphens), en dashes, non-breaking spaces, ellipsis characters, and other Unicode glyphs that can cause JSON parsing failures or unexpected model behavior. This step is critical because the AI response must be parseable as valid JSON, and Unicode artifacts in the prompt can cause the model to produce malformed output. The sanitization is lossless in the sense that the narrative meaning is preserved; only the typographic encoding is normalized.

2.3 JSON Schema Appendage

The system prompt concludes with a complete JSON schema definition that the AI must follow when constructing its response. This schema specifies every field in the `DMResponse` object, including required fields, optional fields, data types, and nested structures. The schema acts as a contract between the AI and the game engine, ensuring that every response can be parsed and applied mechanically. The engine validates the AI response against this schema during the extraction phase, and any field that does not conform is handled by the fallback system described in Chapter 7.

Chapter 3: DM Response Schema

The `DMResponse` is the structured data contract between the AI Dungeon Master and the game engine. Every turn, the AI must produce a JSON object conforming to this schema. The schema contains 20 top-level fields that capture every aspect of the game state update: narrative content, mechanical changes, dice results, and narrative metadata. Understanding this schema is essential for debugging, extending the game, or building custom tools that interact with the engine.

3.1 Narrative Fields

Field	Type	Required	Description
story_summary	string	Yes	One-paragraph summary of what happened this turn
journey_so_far	string	Yes	Updated campaign TLDR (max 150 words)
dm_narration	string	Yes	Full narrative prose (300+ words, 2-4 paragraphs)
tension_note	string	No	DM note on narrative tension level
consequences	string	No	Description of consequences for player choices

Table 3: DMResponse Narrative Fields

The `dm_narration` field is the heart of the player experience. It must contain at least 300 words of rich, atmospheric prose written in a Gaiman-esque literary style. The AI is explicitly instructed to write the narrative first, then append the JSON block. The `story_summary` and `journey_so_far` fields serve as the game memory system, with the summary capturing the immediate turn events and the journey serving as a running campaign synopsis.

3.2 Character & Turn Fields

Field	Type	Description
human_pc_id	string	ID of the PC who acts next in the spotlight
human_pc_reason	string	Narrative justification for why this PC acts next
next_pc_id	string	Alternative next PC (NPC-triggered)
pc_agreement	object[]	Per-PC party consensus rating

Table 4: DMResponse Character & Turn Fields

The `human_pc_id` and `human_pc_reason` fields implement the "spotlight rotation" system, ensuring that different party members receive narrative focus across turns. The AI selects which character should be the protagonist of the next scene and provides a narrative justification. The `pc_agreement` array captures how each party member feels about recent events, enabling the engine to detect intra-party conflict or consensus.

3.3 Combat & Dice Fields

Field	Type	Description
dice_rolls	object[]	Array of d20 rolls with roller name, DC, and success boolean
damage_dealt	object[]	Damage amounts with source and target entity IDs
injury_events	object[]	New injuries with descriptions matched to INJURY_TABLE
state_updates	object[]	HP deltas, conditions, death flags (core update mechanism)

Table 5: DMResponse Combat & Dice Fields

Every d20 roll must be narrated, not merely reported. The AI is instructed to describe the roll dramatically, whether it succeeds or fails. The `damage_dealt` array tracks all damage flows, and the `injury_events` array provides descriptions that the engine fuzzy-matches against the `INJURY_TABLE` to apply mechanical injury effects. The `state_updates` array is the most critical field -- it is the sole mechanism through which the AI can modify the game state, and every change flows through `applyMechanics()` for validation.

3.4 World State Fields

Field	Type	Description
<code>npc_encounters</code>	<code>object[]</code>	NPCs encountered this turn with behavior and pantheon
<code>new_active_npcs</code>	<code>object[]</code>	NPCs added to the active NPC tracking list
<code>shard_event</code>	<code>string</code>	Description of any Shard-related events this turn
<code>boss_phase_trigger</code>	<code>boolean</code>	True if boss combat should advance to next phase
<code>item_drops</code>	<code>object[]</code>	Items discovered or received this turn
<code>quest_updates</code>	<code>object[]</code>	Quest status changes, new objectives, completions

Table 6: DMResponse World State Fields

3.5 State Update Sub-Schema

Each object in the `state_updates` array follows a specific sub-schema. The `pc_id` field is always required and serves as the routing key: it identifies which character or entity the update targets. The remaining fields are optional and specify what changes to apply.

Field	Type	Required	Description
<code>pc_id</code>	<code>string</code>	Yes	Target entity ID ("ANTAGONIST" for boss HP)
<code>hp_delta</code>	<code>number</code>	No	HP change (positive = heal, negative = damage)
<code>new_condition</code>	<code>string</code>	No	Add a status condition (if not already present)
<code>remove_condition</code>	<code>string</code>	No	Remove a specific status condition
<code>dead</code>	<code>boolean</code>	No	If true, mark entity as dead (triggers Test of Faith)

Table 7: State Update Sub-Schema

3.6 Example JSON Response

Below is a simplified example of a valid DMResponse JSON object:

```
{
  "dm_narration": "The chamber trembles as Odin rises...",
  "story_summary": "The party confronted Odin in Asgard...",
  "journey_so_far": "After obtaining the Pale Shard...",
  "human_pc_id": "heracles",
  "human_pc_reason": "Heracles steps forward...",
  "dice_rolls": [
    {"roller": "Heracles", "roll": 17, "dc": 15, "success": true}
  ],
  "state_updates": [
    {"pc_id": "heracles", "hp_delta": -12}
  ],
  "npc_encounters": [{"name": "Odin", "behavior": "hostile"}],
  "shard_event": null,
  "item_drops": [],
  "quest_updates": []
}
```

Chapter 4: State Update Protocol

Every state change in the Deities & Demigods engine flows through the AI JSON response and is processed by the `applyMechanics()` function. This centralized approach ensures that all game state mutations are validated, clamped, and logged consistently. The AI is the sole author of state changes; the player cannot directly modify HP, conditions, or any other mechanical property. This design choice enforces narrative consistency and prevents cheating or state corruption.

4.1 HP Delta Processing

HP changes are the most common state updates. The processing pipeline applies several safeguards to prevent invalid states caused by malformed AI responses:

- **Type Coercion:** Every `hp_delta` value is wrapped in `Number()` to prevent string concatenation bugs. If the AI returns "12" instead of 12, the coercion ensures numeric addition.
- **Upper Clamp:** HP is capped at `maxHp` using `Math.min(maxHp, currentHp + delta)`. A healing potion cannot raise HP above the character maximum.
- **Lower Clamp:** HP is clamped to 0 minimum. HP cannot go negative, even from massive damage.
- **Antagonist Routing:** If `pc_id` equals "ANTAGONIST", the `hp_delta` is routed to `antagonistHp` instead of any character object. Antagonist HP uses separate `maxHp` tracking.

4.2 Condition Management

Status conditions (such as "poisoned", "stunned", "invisible", "blessed") are managed as string arrays on each character object. The `applyMechanics()` function processes two condition-related fields from the `state_updates`:

- **new_condition:** Added to the character condition array only if not already present. This prevents duplicate conditions from stacking unintentionally.
- **remove_condition:** Filtered out of the character condition array. If the condition does not exist, the removal is silently ignored (no error thrown).
- Both operations are case-sensitive. The AI must use the exact condition string defined in the game constants to ensure proper matching.

4.3 Death Detection & Processing

Death is detected through two mechanisms. First, if the AI explicitly sets `dead: true` in a state update, the character is immediately marked as deceased. Second, if any HP update results in HP reaching 0, the engine automatically sets `dead: true` and clamps HP to 0. Upon death detection, several cascading systems activate:

1. The dead flag is set to true and HP is clamped to 0.
2. If the deceased PC was carrying the prophecy, the prophecy is transferred to another living PC (the "successor" mechanism).
3. The Test of Faith engine is triggered (see Chapter 8) to determine if divine intervention occurs.
4. The character is excluded from the Party State block in future prompts, preventing the AI from referencing them as active participants.
5. All conditions on the deceased character are preserved (for potential revival mechanics).

4.4 Antagonist HP Routing

The antagonist (final boss) is not part of the regular party array and therefore requires a separate routing mechanism. When the AI returns a `state_update` with `pc_id` set to the string "ANTAGONIST", the engine routes the `hp_delta` to the `antagonistHp` field on the `GameState` object. Antagonist HP is clamped between 0 and `antagonistMaxHp` using the same `Math.min/Math.max` safeguards applied to player characters. When `antagonistHp` reaches 0, the boss is considered defeated and the victory conditions are evaluated.

4.5 NPC State Updates

If the `pc_id` in a `state_update` does not match any character in the party, the engine searches the `activeNPCs` array for a matching NPC name or ID. If found, the HP delta and condition updates are applied to that NPC. If an NPC is marked as dead, they are removed from the `activeNPCs` list entirely, preventing them from appearing in future encounters or state snapshots.

4.6 Save/Load State Merging

The save system uses `localStorage` for persistence. When a saved game is loaded, the `loadGame()` function merges the saved state with the defaults from `createInitialState()`. This merge strategy handles version upgrades gracefully: if a newer version of the game adds new fields to the state object, loading an older save

file will populate those fields with their default values rather than leaving them undefined. Missing fields are filled with defaults, and the game continues from the last valid state without errors.

Chapter 5: Success Rate Engine

The success rate is the single most important metric in the Deities & Demigods engine. It represents the party overall probability of defeating the antagonist and completing the campaign victoriously. Displayed to the player as a percentage, it is recalculated after every turn once all state updates have been applied. The success rate is not a static difficulty setting but rather an emergent property of the party current status, achievements, and narrative progress.

5.1 The Nine Factors

The `calculateSuccessRate(factors)` function computes the final percentage by summing nine distinct factors, each representing a different aspect of party strength or narrative advantage. The formula is designed to produce a baseline of approximately 50% for a fresh party, with meaningful variation based on player decisions and campaign events.

#	Factor	Formula	Range	Description
1	Base Chance	50 (fixed)	+50	Starting baseline for all campaigns
2	Party Living	$\min(\text{livingPCs} * 2, 10)$	+0 to +10	+2 per living PC, max 10
3	Prophecy State	See mapping	-5 to +8	dormant +0, awakening +3, manifesting +5, fulfilled +8, broken -5
4	Allied Gods	$\min(\text{alliedGods} * 3, 15)$	+0 to +15	+3 per allied good NPC, max 15
5	PC Renown	$\min(\text{Sum}(\text{level}/3), 8)$	+0 to +8	Sum of $\text{classLevel}/3$ per PC
6	Power Bonus	$\min(\text{Sum}(\text{hp}/100), 10)$	+0 to +10	Sum of $\text{currentHP}/100$ per PC
7	Alignment Harmony	$\text{clamp}(\text{harmony}, -5, +5)$	-5 to +5	Party alignment agreement score
8	Story Achievements	$\min(\text{achievements} * 2, 12)$	+0 to +12	Completed quests + $\text{floor}(\text{clues}/2)$
9	Antagonist Type	$\text{greater_god} ? -5 : 0$	-5 or 0	Penalty for Greater God antagonists

Table 8: Success Rate Factor Breakdown

5.2 Total Clamping

After all nine factors are summed, the total is clamped to the range [5, 95]. This prevents the success rate from reaching 0% (which would make the game unwinnable) or 100% (which would remove all tension). The clamping ensures that there is always a meaningful chance of both victory and defeat, maintaining player engagement throughout the campaign. In practice, early-game success rates typically hover around 50-65%, mid-game around 40-60%, and end-game shows the widest variance at 25-75% depending on the party cumulative achievements and losses.

5.3 Factor Recalculation Details

Each factor is computed from live game state data on every recalculation. The key computations are:

- **livingPCs:** `party.pcs.filter(pc => !pc.dead).length` -- counts only surviving characters.
- **alliedGods:** `npcHistory.filter(npc => npc.alignment and npc.alignment.includes("good")).length` -- counts all NPCs with "good" in their alignment string.
- **pcRenown:** Sum of `Math.max(Math.floor(pc.classLevel / 3))` across all living PCs. Includes `thiefLevel` as a separate component for thief-class characters.
- **pcPower:** Sum of `(pc.hp / 100)` across all living PCs, reflecting raw survivability.
- **storyAchievements:** `completedQuests.length + Math.floor(clues.length / 2)`. Both quest completion and clue discovery contribute to the party narrative momentum.

Chapter 6: Narrative Voice Guidelines

The narrative voice of the Deities & Demigods AI Dungeon Master is deliberately crafted to evoke the style of Neil Gaiman -- literary, atmospheric, wry, and deeply aware of the weight of myth. This is not the breezy tone of a typical chatbot nor the dry exposition of a technical manual. The DM speaks with the authority of someone who has witnessed the birth and death of gods and understands that mythology is not merely story but the architecture of belief itself.

6.1 Prose Requirements

- **Minimum Length:** `dm_narration` must contain at least 300 words per turn (2-4 rich paragraphs). Shorter responses are flagged by the validation system.
- **Sensory Detail:** Every scene must include at least two distinct sensory impressions -- the smell of ozone before a lightning bolt, the texture of ancient stone under trembling fingers, the sound of a god laughter echoing through empty halls.
- **Character Voice:** Each NPC speaks with a distinct voice. Odin speaks in riddles; Thor bellows; Loki whispers with honeyed words. The AI must maintain these vocal signatures across turns.
- **Mythological Weight:** References to the deeper mythology of each pantheon should be woven naturally into the narrative, not dumped in exposition blocks.

6.2 Dice Roll Narration

Every d20 roll must be narrated, not merely reported. The AI is instructed to describe the physical action of the roll, the moment of tension as the die tumbles, and the dramatic outcome. A natural 20 should feel like a moment of divine favor -- the blade finding the one gap in the armor, the words striking exactly the right chord. A natural 1 should feel like fate conspiring against the hero -- the ground giving way at the worst possible moment, the spell sputtering out just as it needed to work. Even mundane successes and failures should carry narrative weight.

6.3 Injury & Death Narration

Injuries are narrated with visceral, sensory detail. A deep cut is not "you take 8 damage" but rather a description of steel parting flesh, the hot rush of blood, the shock of pain that makes the world narrow to a single point. Death is treated with gravity and weight -- it is permanent in this game, and the narration must reflect that permanence. The loss of a character should feel like the loss of a mythic figure: profound, irreversible, and resonant with meaning.

6.4 Token Budgets

Context	Token Budget	Purpose
Opening Scene	8,000 tokens	Full establishing narrative with character introductions
Regular Turn	4,000 tokens	Ongoing narrative progression and mechanical updates
Action Options	200 tokens	Concise player choice descriptions (via Groq)

Table 9: Narrative Token Budgets

The system prompt explicitly instructs the AI: "Write the narrative prose first. Then, append the JSON block at the end." This ordering ensures that the AI prioritizes storytelling quality over mechanical bookkeeping, producing prose that reads naturally without JSON artifacts.

Chapter 7: Fallback System

No AI system is infallible. Network connections drop, rate limits trigger, API keys expire, and sometimes the model simply produces malformed output. The Deities & Demigods engine implements a comprehensive fallback system that handles six distinct failure scenarios, ensuring that the game never becomes unplayable due to a transient error. Each scenario has been designed to be recoverable except for the catastrophic case of an invalid API key.

7.1 Scenario 1: API Rate Limit (HTTP 429)

When the Gemini API returns a 429 status code, the engine initiates exponential backoff with three retry attempts. The delays are 6 seconds, 12 seconds, and 24 seconds respectively. If all three retries fail, the engine falls back to a pre-written template that generates a generic scene progression. The template is context-aware -- it references the current act, location, and party status to maintain narrative continuity even without AI generation.

Recovery: Automatic. The next player action will retry the API call with a fresh request.

7.2 Scenario 2: Malformed JSON Response

The most common failure mode is the AI producing JSON that does not perfectly conform to the expected schema. The engine handles this through a multi-stage recovery process:

1. **Regex Extraction:** The engine uses a regex pattern to locate the JSON block within the AI response, handling markdown code fences, inline JSON, and various formatting edge cases.
2. **Partial Parse:** If the JSON is partially valid (some fields missing or malformed), the engine extracts whatever fields are available and fills missing fields with safe defaults.
3. **Narration Salvage:** The `dm_narration` field is always the highest priority. Even if the JSON is completely unparseable, the engine attempts to extract narrative text from the response body and displays it to the player.
4. **Default State:** If no `state_updates` can be parsed, the engine assumes no state changes occurred this turn. The game continues with the previous state intact.

7.3 Scenario 3: Empty/Truncated Response

Sometimes the AI returns an empty response or a response truncated by token limits. In these cases, the engine falls back to a library of pre-written scene templates categorized by act and narrative situation. Each template contains 200-400 words of generic but atmospheric prose that references the party and current quest context. The template system is designed to be invisible to the player -- the prose reads naturally and maintains narrative momentum.

7.4 Scenario 4: Invalid API Key / Network Error

This is the only non-recoverable failure scenario. If the API key is invalid or the network is completely unavailable, the engine displays an error message to the player explaining that the AI DM cannot be reached. The game state is preserved, and the player can retry after resolving the issue. This scenario is marked as BLOCKING because no template can substitute for the core AI narrative generation.

Recovery: BLOCKING. Player must fix the API key or network connection to continue.

7.5 Scenario 5: Save Data Corruption

Save game data stored in localStorage can theoretically become corrupted by browser storage limits, manual editing, or extension interference. The loadGame() function handles this by merging the saved data with the defaults from createInitialState(). Any missing or malformed field is replaced with its default value. This means that even a partially corrupted save can be recovered, though some progress (such as items or quest state in the corrupted fields) may be lost.

Recovery: RECOVERABLE. Missing fields are filled with defaults; game continues.

7.6 Scenario 6: Injury ID Not Found

When the AI generates an injury_event with a description that does not exactly match any entry in the INJURY_TABLE, the engine performs fuzzy matching against the table using string similarity scoring. If no match exceeds the similarity threshold, the engine assigns a generic injury from the most appropriate category (Physical, Magic, Poison, or Psionic) based on keyword analysis of the description. This ensures that every injury described in the narrative produces a mechanical effect, even if the AI uses creative or unusual terminology.

Recovery: RECOVERABLE. Fuzzy matching finds closest injury; fallback to generic.

7.7 Post-Parse Validation

After all extraction and parsing is complete, the engine performs a final validation check. The dm_narration field must exist and must be a non-empty string. If this validation fails, the engine uses a last-resort fallback template that apologizes for the interruption and provides a minimal scene description. This ensures the player always receives some narrative content, even in the worst-case failure scenario.

Chapter 8: Test of Faith Engine

The Test of Faith is one of the most dramatic mechanical systems in Deities & Demigods. It represents moments of divine intervention where the gods themselves intervene in mortal affairs -- sometimes to aid the party, sometimes to hinder them, and sometimes to simply watch. The system adds an element of unpredictability and mythological resonance that elevates the game beyond a pure stat-check simulation.

8.1 Trigger Conditions

The checkTestOfFaith() function runs after every applyMechanics() call. It evaluates a series of guard conditions and trigger conditions in strict order. All guard conditions must pass before any trigger condition is evaluated.

Guard Conditions

1. **Act Guard:** Must be Act II or later. Tests of Faith do not occur during the tutorial phase of Act I.

2. Cooldown Guard: At least 10 turns must have passed since the last Test of Faith. This prevents divine intervention from feeling routine or spammy.

3. Miracle Threshold: Fewer than 2 miracles must have been used in the entire campaign. This ensures that miracles remain rare and dramatic.

Trigger Conditions (checked if all guards pass)

1. Death Save: A PC died this turn and has not been previously saved by a miracle. This is the most common trigger -- a dying character calls out for divine aid.

2. Boss Phase: The AI set boss_phase_trigger to true, indicating the antagonist is entering a new and more dangerous combat phase.

3. Desperate Odds: The success rate has dropped below 40%, indicating the party is in serious danger of campaign failure.

8.2 Roll Processing

When a Test of Faith is triggered, the system rolls a d20 and processes the result through three outcome brackets. Each bracket produces dramatically different mechanical effects.

Roll Range	Outcome	Mechanical Effects
18-20	MIRACLE	+8 success rate, +1 shard charge, rekindle shard. Death: revive at 1 HP, restore prophecy, remove successor. Boss: 15% max HP damage.
4-17	FATE HOLDS	No mechanical changes. The gods watch but do not act.
1-3	MURPHY'S LAW	-5 success rate, random item loses 1 charge, shard darkens.

Table 10: Test of Faith Outcome Table

8.3 Miracle Effects in Detail

A Miracle (roll 18-20) is the most powerful single event in the game. It combines narrative dramatics with significant mechanical benefits:

- **Success Rate Boost:** +8 percentage points, representing divine favor improving the party overall odds.
- **Shard Recharge:** +1 charge to the Shard artifact, allowing an additional summoning attempt.
- **Shard Rekindle:** If the Shard was darkened (from a previous Murphy Law result), it is restored to normal status. The shardDark flag is set to false.
- **Death Reversal:** If the trigger was a PC death, the deceased character is revived at 1 HP. If the prophecy had been transferred to a successor PC, the successor is removed and the original carrier reclaims the prophecy.
- **Boss Damage:** If the trigger was a boss phase transition, the antagonist takes damage equal to 15% of their maximum HP, representing divine wrath interfering with the boss power-up.

8.4 Prophecy Restoration

When a Miracle reverses a death that caused a prophecy transfer, the engine must carefully unwind the succession chain. If a successor PC was added solely as a prophecy carrier (i.e., they were not already in the party for other reasons), they are removed from the party roster. The original PC reclaims their role as prophecy carrier, and the prophecy state is restored to whatever it was before the death occurred. This restoration ensures narrative consistency -- the prophecy follows its destined bearer, not an emergency replacement.

Chapter 9: Difficulty Scaling

Difficulty in Deities & Demigods is not controlled by a single slider or setting. Instead, it emerges from the complex interaction of party composition, narrative choices, resource management, and random chance. The success rate formula (Chapter 5) produces different difficulty curves at different stages of the campaign, and the AI DM is instructed to calibrate narrative tension based on the current success rate value.

9.1 Difficulty Curve by Act

Act	Typical Success Rate	Characteristics
Act I	50-65%	Party forming, no major threats, prophecy dormant, full party alive
Act II	40-60%	Antagonist -5 penalty active, injuries accumulate, NPCs may be lost
Act III	25-75%	Boss battle, high variance, miracles possible, final confrontation

Table 11: Difficulty Curve by Campaign Act

9.2 Difficulty-Increasing Factors

- **PC Deaths:** Each death removes the +2 living PC bonus and reduces the party capability. Multiple deaths compound this effect.
- **Broken Prophecy:** The -5 penalty from a broken prophecy significantly impacts the success rate and represents the party failure to fulfill a divine mandate.
- **Alignment Disharmony:** When party members have conflicting alignments, the harmony bonus becomes a penalty, reducing the success rate by up to 5 points.
- **Greater God Antagonist:** The flat -5 penalty for facing a Greater God reflects the immense power differential between mortals and supreme deities.
- **Injury Accumulation:** Injuries reduce effective HP and combat capability, indirectly lowering the Power Bonus factor.

9.3 Difficulty-Decreasing Factors

- **Allied NPCs:** Each good-aligned NPC in the history adds +3 to the success rate, up to a maximum of +15. Diplomatic play is rewarded mechanically.
- **Fulfilled Prophecy:** Achieving a fulfilled prophecy state grants +8 to the success rate, the single largest positive bonus available.
- **Completed Quests:** Each completed quest contributes to the Story Achievements factor, rewarding thorough exploration and engagement.
- **High PC Levels:** Characters with higher class levels contribute more to the PC Renown factor, representing growing legendary status.

Chapter 10: HP Validation & Combat Safety

HP management is one of the most safety-critical systems in the engine. Because the AI generates HP deltas as part of its JSON response, and because AI output can be unpredictable, the engine implements multiple layers of validation to prevent invalid HP states. These safeguards protect against both accidental errors (the AI miscounting damage) and edge cases (simultaneous death and healing effects).

10.1 HP Capping

- **Upper Cap:** HP is capped using `Math.min(maxHp, currentHp + delta)` for all entity types. No healing effect can raise HP above the entity maximum. This applies to PCs, NPCs, and antagonists.
- **Lower Cap:** HP is clamped to 0 using `Math.max(0, newHp)`. HP cannot go negative, even from massive damage. If a character has 5 HP and takes 50 damage, HP becomes 0 (not -45).
- **Antagonist Capping:** Antagonist HP uses the same `Math.min/Math.max` pattern with `antagonistMaxHp` as the upper bound.

10.2 HP Delta Coercion

Every `hp_delta` value from the AI response is wrapped in `Number()` before processing. This prevents string concatenation bugs that could occur if the AI returns a string representation of a number (e.g., "-12" instead of -12). Without this coercion, the expression `currentHp + "-12"` would produce string concatenation rather than subtraction, resulting in invalid HP values like "45-12" instead of 33.

10.3 Damage Over Time (DOT)

Conditions that cause ongoing damage (such as poison or bleeding) are processed at the start of each turn. The DOT check evaluates whether hp has dropped to 0 or below and, if so, sets the dead flag to true. This ensures that damage-over-time effects can kill characters, not just damage them. Death from DOT triggers the same cascade as death from direct damage: prophecy transfer, Test of Faith check, and removal from the

active party block.

10.4 Death Cascade

When any entity HP reaches 0 (whether from direct damage, DOT, or a `state_update` with `dead: true`), the following cascade is triggered:

1. The dead flag is set to true and HP is clamped to 0.
2. For PCs carrying the prophecy: the prophecy is transferred to a successor.
3. The Test of Faith engine is invoked to check for divine intervention.
4. If no miracle occurs, the character is permanently removed from active play.
5. For NPCs: the entity is removed from the `activeNPCs` array.
6. For the antagonist: victory conditions are evaluated.

Chapter 11: Item System Internals

The item system in Deities & Demigods is more complex than a simple inventory list. Items have active effects (triggered on use), passive modifiers (applied on equip), charges (consumed on use), and rarity tiers that influence drop rates and AI behavior. The system supports 35 distinct items across four categories, each with unique mechanical effects.

11.1 Item Template Structure

All items are defined in the `ITEM_TEMPLATES` constant, which contains 35 complete item definitions. Each template specifies the item name, category, rarity, description, active modifiers, passive modifiers, maximum charges, and any special conditions. The AI references these templates when generating `item_drops` in its response JSON.

11.2 Active Modifiers

Active modifiers are processed by the `handleUseItem()` function when a player uses an item from their inventory. The following modifier types are supported:

Modifier	Effect	Example Items
healing	Restore specified HP amount	Healing Potion, Greater Healing Potion
full_heal	Restore to max HP	Ambrosia
cure_poison	Remove poison conditions	Antitoxin, Universal Antidote
cure_all_poison	Remove all injuries in poison category	Universal Antidote

Modifier	Effect	Example Items
death_ward	Prevent next death	Aegis Fragment
invisible	Apply invisible condition	Potion of Invisibility
str_set	Set STR to specified value	Potion of Giant Strength

Table 12: Active Item Modifiers

11.3 Passive Modifiers

Passive modifiers are applied when an item is equipped and remain active until the item is unequipped or its charges are depleted. Passive modifiers include:

- **damage:** Adds flat damage bonus to all attacks.
- **ac:** Improves Armor Class (lower is better).
- **regen:** Restores a fixed HP amount at the start of each turn.
- **vampiric:** Heals the wielder for a percentage of damage dealt.
- **fear_immune:** Prevents fear-based conditions and morale checks.
- **true_sight:** Reveals invisible entities and illusions.

11.4 Deep Copy Safety

A critical implementation detail: the `cure_all_poison` modifier deep copies the `injuries` array before filtering out poison-type injuries. This prevents cache corruption that could occur if the engine mutated the `injuries` array directly. Without the deep copy, reference sharing between the game state and the React component state could cause phantom injuries to appear or disappear between renders. This is a common pitfall in JavaScript state management, and the deep copy ensures that each state mutation produces a clean, independent object graph.

11.5 Charge System

Items with limited charges track their remaining uses in the `charges` field. Each use decrements the charge count by 1. When charges reach 0, the item remains in inventory but becomes unusable. The AI generates `item_drops` in its response JSON, and the engine adds new items (with full charges) to the appropriate party member inventory. Certain powerful items have only 1 charge, making each use a significant strategic decision.

Chapter 12: Quest System

The quest system tracks narrative objectives that provide structure to the campaign and contribute to the success rate calculation. Quests are managed as structured objects with well-defined states, and the AI DM generates `quest_updates` as part of every turn response to advance or modify quests.

12.1 Quest Object Structure

Each quest is represented by a Quest object containing the following fields:

- **id:** Unique string identifier for the quest.
- **title:** Display name shown in the quest log.
- **description:** Full narrative description of the quest objectives.
- **type:** One of "main", "side", or "hidden". Main quests drive the central plot; side quests offer optional rewards; hidden quests are revealed through exploration.
- **status:** One of "active", "completed", or "failed". Only active quests appear in the system prompt for DM awareness.
- **objectives:** Array of specific, measurable objectives that must be completed.
- **rewards:** Description of rewards granted upon completion.

12.2 Starting Quests

Every new campaign begins with two quests automatically added to the quest log:

- **"The Shard Awakens" (main):** The central quest that drives the entire campaign. Objectives include discovering the Shard, understanding its power, and using it against the antagonist.
- **"Divine Allies" (side):** A side quest encouraging the party to seek out and befriend NPCs, particularly good-aligned deities who can contribute to the success rate.

12.3 Quest Updates

The AI generates `quest_updates` in its response JSON each turn. These updates can create new quests, advance existing quest objectives, complete quests, or mark quests as failed. The `quest_updates` are included in the system prompt schema so the AI DM is always aware of the current quest landscape and can make narrative decisions that align with active objectives.

Chapter 13: Save/Load Architecture

The save/load system provides persistent storage for campaign progress using the browser `localStorage` API. The system is designed for reliability and forward compatibility, handling version upgrades gracefully by merging saved data with default state templates.

13.1 Save Slots

The game supports multiple save slots, each represented by a `SaveSlot` interface:

- **id:** Unique identifier for the save slot.
- **name:** Player-assigned name for the save.
- **timestamp:** ISO date string of when the save was created.
- **turn:** The game turn number at the time of saving.
- **act:** The current act (1, 2, or 3).
- **partyNames:** Array of party member names for quick identification.

13.2 Save Mechanism

When the player saves (manually or via auto-save), the complete `GameState` object is serialized to JSON and stored in `localStorage` under a key derived from the save slot ID. The serialization includes all mutable game state: party data, antagonist HP, quest status, prophecy state, NPC history, conversation history, Shard status, items, injuries, and the journey summary.

13.3 Load Mechanism

When loading a saved game, the `loadGame()` function performs a merge operation between the saved state and the defaults from `createInitialState()`. This merge follows a simple rule: for every field in the default state, if the saved state contains a value for that field, the saved value is used; otherwise, the default value is applied. This approach provides several benefits:

- **Version Upgrade Compatibility:** New fields added in later game versions are populated with defaults when loading older saves.
- **Corruption Recovery:** Missing or undefined fields do not cause crashes; they are silently replaced with safe defaults.
- **Debugging Support:** Developers can manually edit `localStorage` entries to test specific game states without worrying about incomplete data.

The load operation is atomic: if any error occurs during deserialization, the system falls back to `createInitialState()` and starts a fresh game rather than crashing. This ensures that the game is always launchable, even if saved data is corrupted beyond recovery.